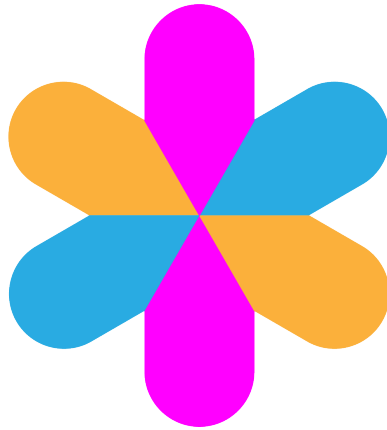




Capstone Project 3

Vizuara 5D Parallelism Workshop

Training a Small Language Model
with Five Dimensions of Parallelism

Data Parallelism • Tensor Parallelism • Pipeline
Parallelism
Context Parallelism • Expert Parallelism

Based on Andrej Karpathy's `build-nanogpt`

<https://github.com/karpathy/build-nanogpt>

Total Points: 140
Estimated Time: 30–35 hours
Hardware: 8× GPU Pod (H100/A100 SXM 80GB)
Architecture: GPT-2 124M (12 layers, 12 heads, 768 dim)
Dataset: FineWeb-Edu (10B tokens)
Code Repository: <https://github.com/VizuaraAI/vizuara-5d-parallelism-workshop>

Welcome to the 5D Parallelism Workshop

You are about to embark on one of the most ambitious and rewarding projects in this course: building a language model training system that harnesses **all five dimensions of parallelism** used in state-of-the-art AI infrastructure.

The same techniques you will implement here—tensor parallelism, pipeline parallelism, ring attention, mixture of experts—are the exact methods used to train GPT-4, Llama 3, DeepSeek-V3, and every other frontier model. This is not a toy exercise. **This is real distributed systems engineering.**

Imagine you are the founding ML engineer at **Vizz AI**, a startup building the first high-quality Small Language Model for the Vizz language—spoken by **850 million people** but virtually absent from every major LLM.

You have collected 500 billion tokens from local publishers, radio transcripts, government records, and web scrapes. Your training data includes documents up to 128K tokens long—legal proceedings, parliamentary debates, epic folklore. Your target: a 7 billion parameter Mixture-of-Experts transformer.

Your cluster of 8 GPUs is ready. But a single GPU can't:

- Hold the full 7B model weights in memory
- Process 128K-token sequences (attention is $O(T^2)$ in memory)
- Train fast enough—you need every GPU working in concert

You need to master **all five dimensions of parallelism**. In this workshop, you will implement each one from scratch, using PyTorch's distributed primitives, and compose them together to train on all 8 GPUs simultaneously.

For practical reasons, we use the **GPT-2 124M architecture** as our testbed (the same architecture from Karpathy's `build-nanogpt`). The parallelism techniques you implement will work identically at 7B, 70B, or 700B scale—only the config changes.

The skeleton code is a modular refactor of Karpathy's single-file `train_gpt2.py`. It **runs out-of-the-box with Data Parallelism on all 8 GPUs**. Your job is to implement the other four dimensions on top.

Let's build something extraordinary.

Overview

The Five Dimensions of Parallelism

1. **Data Parallelism (DP)** — Replicate the model on each GPU, split the training data. Gradients are synchronized via all-reduce. *Already implemented in the skeleton.*
2. **Tensor Parallelism (TP)** — Split individual layers (attention heads, FFN intermediate dimension) across GPUs. Each GPU computes its shard of every matrix multiply.
3. **Pipeline Parallelism (PP)** — Split the model depth-wise into stages. GPU 0 runs layers 0–5, GPU 1 runs layers 6–11. Data flows through stages as micro-batches.
4. **Context Parallelism (CP)** — Split long sequences across GPUs. Each GPU holds a chunk of Q/K/V. Ring Attention rotates K/V around a ring so every query attends to all keys.
5. **Expert Parallelism (EP)** — In Mixture-of-Experts models, distribute expert FFNs across GPUs. Tokens are dispatched to the GPU holding their assigned expert via all-to-all communication.

The Fundamental Constraint

$$TP \times PP \times CP \times EP \times DP = \text{Total GPUs (8)}$$

Every GPU must belong to exactly one group in each dimension. The product of all group sizes must equal the world size.

Code Structure

```
vizz_slim/
+-- config.py          # ModelConfig, ParallelConfig, TrainConfig
+-- model.py           # GPT architecture (refactored from nanogpt)
+-- data.py            # DataLoaderLite
+-- train.py           # Main training loop with DDP
+-- parallel/
|   +-- tensor.py      # TODO: ColumnParallelLinear, RowParallelLinear
|   +-- pipeline.py    # TODO: PipelineStage, 1F1B schedule
|   +-- context.py     # TODO: Ring Attention
|   +-- expert.py      # TODO: TopKRouter, MoELayer
+-- utils.py           # TODO: create_process_groups
+-- tests/             # Correctness tests for each module
+-- run.sh             # Launch script
```

1 Part 0: Setup & Baseline

[10 points]

Learning Objectives

- Set up a multi-GPU RunPod environment
- Understand the GPT-2 architecture and training loop
- Establish single-GPU throughput baselines

Tasks

0.1 Clone the skeleton repository and set up the environment:

```
# On your RunPod pod (8x H100 SXM or 8x A100 SXM)
pip install tiktoken datasets tqdm
git clone https://github.com/VizuaruAI/vizuaru-5d-parallelism-workshop.git \
    vizz_slim && cd vizz_slim
python fineweb.py # Downloads ~20GB, takes ~30-60 min
```

0.2 Run single-GPU training for 100 steps:

```
python train.py --max_steps 100
```

Record: (a) final training loss, (b) tokens/sec throughput, (c) peak GPU memory (`nvidia-smi`).

0.3 Answer these questions:

- How many parameters does the GPT-2 124M model have? Calculate manually from the architecture ($n_{layer} = 12$, $n_{head} = 12$, $n_{embd} = 768$, $vocab_size = 50304$).
- What is the total batch size in tokens per step with $B = 64$, $T = 1024$?
- Why is `vocab_size` padded to 50304 (from 50257)? How does this help GPU performance?

Grading

Criteria	Points
Environment setup verified (<code>nvidia-smi</code> shows 8 GPUs)	4
Correct throughput measurement + loss curve screenshot	3
Conceptual questions answered correctly	3

2 Part 1: Data Parallelism Deep Dive

[15 points]

Learning Objectives

- Understand how DDP synchronizes gradients (all-reduce)
- Measure strong scaling efficiency across 1–8 GPUs
- Understand gradient accumulation and its interaction with DDP

Background

Distributed Data Parallel (DDP) replicates the model on each GPU. Each GPU processes a different mini-batch. After the backward pass, gradients are averaged across GPUs via an **all-reduce** operation. The skeleton code already implements DDP via `torchrun`.

A key optimization: `model.require_backward_grad_sync` controls whether DDP synchronizes gradients during each micro-step. Setting it to `False` for all but the last micro-step avoids redundant all-reduce operations during gradient accumulation.

Tasks

1.1 Run training with DDP on 1, 2, 4, and 8 GPUs for 50 steps each:

```
# 1 GPU
python train.py --max_steps 50
# 2 GPUs
torchrun --standalone --nproc_per_node=2 train.py --max_steps 50
# 4 GPUs
torchrun --standalone --nproc_per_node=4 train.py --max_steps 50
# 8 GPUs
torchrun --standalone --nproc_per_node=8 train.py --max_steps 50
```

Plot throughput (tok/sec) vs. number of GPUs. Calculate strong scaling efficiency: $\text{efficiency} = \frac{\text{throughput}(N)}{N \times \text{throughput}(1)}$.

1.2 In `train.py`, find the gradient synchronization control:

```
model.require_backward_grad_sync = (micro_step == grad_accum_steps - 1)
```

Explain: Why is this optimization important? What happens if you set it to `True` always?

1.3 Answer these questions:

- What collective operation does DDP use for gradient sync? (Name and describe it)
- For a 124M parameter model in `bfloat16`, how many bytes are communicated per all-reduce?
- At what GPU count does communication bandwidth become the training bottleneck?

Grading

Criteria	Points
Scaling plot with 1/2/4/8 GPUs + efficiency calculation	5
Gradient sync optimization explanation	5
Written analysis of communication overhead	5

3 Part 2: Tensor Parallelism

[25 points]

Learning Objectives

- Understand column-parallel and row-parallel linear layer decomposition
- Implement TP for attention (splitting heads) and MLP (splitting intermediate dim)
- Understand the all-reduce communication pattern in TP

Background

Tensor Parallelism splits individual matrix multiplications across GPUs:

ColumnParallelLinear: Splits the *output* dimension. The weight matrix $W \in R^{d_{out} \times d_{in}}$ is partitioned row-wise: each GPU holds $W_i \in R^{(d_{out}/P) \times d_{in}}$. Each GPU computes $Y_i = XW_i^T$ —a shard of the output. No communication needed for the matmul itself.

RowParallelLinear: Splits the *input* dimension. The weight matrix is partitioned column-wise: each GPU holds $W_i \in R^{d_{out} \times (d_{in}/P)}$. Each GPU computes a partial result $Y_i = X_iW_i^T$, then an **all-reduce** sums the partials: $Y = \sum Y_i$.

These naturally pair: ColumnParallel feeds RowParallel without communication. Only one all-reduce per pair is needed (in RowParallel's forward). In a transformer block, this gives **2 all-reduces per layer**:

```
Attention: c_attn (Column) -> local attention -> c_proj (Row + all-reduce)
MLP:      c_fc (Column) -> GELU -> c_proj (Row + all-reduce)
```

Tasks

2.1 Implement `ColumnParallelLinear.forward()` in `parallel/tensor.py`:

- Compute local matmul: `y_local = F.linear(x, self.weight, self.bias)`
- If `gather_output=True`, all-gather across TP ranks using `dist.all_gather()`

2.2 Implement `RowParallelLinear.forward()` in `parallel/tensor.py`:

- Compute local matmul **without bias**: `y_partial = F.linear(x, self.weight)`
- All-reduce to sum partials: `dist.all_reduce(y_partial, op=SUM, group=tp_group)`
- Add bias **after** all-reduce (to avoid double-counting)

2.3 Implement `apply_tensor_parallelism()` in `model.py`:

- Replace `c_attn` with `ColumnParallel`, `c_proj` with `RowParallel` in attention
- Replace `c_fc` with `ColumnParallel`, `c_proj` with `RowParallel` in MLP
- Update `n_head` to `n_head // tp_size` for local head computation
- Copy the appropriate weight slices from the original layers to the new ones

2.4 Verify correctness and measure performance:

```
# Test correctness
torchrun --standalone --nproc_per_node=2 tests/test_tp.py

# Compare DP=8 vs TP=2+DP=4
torchrun --standalone --nproc_per_node=8 train.py --max_steps 50
```

```
torchrun --standalone --nproc_per_node=8 train.py --tp_size 2 --max_steps 50
```

Compare: throughput (tok/sec), per-GPU memory, final loss at step 50.

The fused QKV projection (`c_attn`) outputs $3 \times n_{embd}$. With ColumnParallel, each rank gets $3 \times n_{embd}/tp_size$, naturally giving each rank its share of Q, K, V heads.

After replacing `c_attn`, `n_head` must be updated to n_head/tp_size for the view/reshape in attention.

Bias in RowParallel must be added **after** all-reduce. Adding before causes $\text{bias} \times tp_size$.

Grading

Criteria	Points
<code>ColumnParallelLinear.forward()</code> correct	8
<code>RowParallelLinear.forward()</code> correct	8
Attention + MLP integration (<code>apply_tensor_parallelism</code>)	5
Performance comparison and analysis	4

4 Part 3: Pipeline Parallelism

[25 points]

Learning Objectives

- Understand how to partition a model into pipeline stages
- Implement micro-batching with the 1F1B schedule
- Measure and understand pipeline bubble overhead

Background

Pipeline Parallelism divides the model vertically by layers. Data flows through stages sequentially. To avoid GPU idle time, we split each mini-batch into micro-batches and pipeline them.

1F1B Schedule (One-Forward-One-Backward):

- **Warmup:** Forward passes to fill the pipeline ($PP - 1$ forwards)
- **Steady state:** Alternate 1 forward + 1 backward
- **Cooldown:** Drain remaining backward passes

The pipeline bubble fraction = $(PP - 1)/\text{num_micro_batches}$. With $PP = 2$ and 8 micro-batches, only 12.5% of compute is wasted in the bubble.

Communication: Point-to-point sends/receives between adjacent stages (activations forward, gradients backward).

Tasks

3.1 Implement `create_pipeline_stages()` in `parallel/pipeline.py`:

- Divide `n_layer` layers evenly across `num_stages`
- First stage holds embeddings (`wte`, `wpe`), last stage holds `ln_f` + `lm_head`
- Return the `PipelineStage` for this rank

3.2 Implement `pipeline_schedule_1f1b()` in `parallel/pipeline.py`:

We recommend implementing in two phases:

- **Phase A:** Naive schedule (all forwards, then all backwards)—get correctness first
- **Phase B:** Convert to 1F1B—optimize for GPU utilization

Key communication primitives:

```
# Send activations to next stage
dist.send(hidden.contiguous(), dst=next_rank)

# Receive activations from previous stage
recv_buf = torch.empty(shape, device=device)
dist.recv(recv_buf, src=prev_rank)
recv_buf.requires_grad_(True) # Enable gradient flow
```

3.3 Test and measure:

```
torchrun --standalone --nproc_per_node=2 tests/test_pp.py
torchrun --standalone --nproc_per_node=8 train.py --pp_size 2 --max_steps 50
```

Deadlock: Ensure send/recv pairs match across stages (stage 0 sends while stage 1 receives).

`recv_buf` must have `requires_grad=True` for backward to flow through it.

After backward, `recv_buf.grad` is the gradient to send back to the previous stage.

Grading

Criteria	Points
<code>create_pipeline_stages()</code> correct layer assignment	8
Pipeline schedule with correct forward/backward	12
Bubble fraction analysis	5

5 Part 4: Context Parallelism

[20 points]

Learning Objectives

- Understand how to split long sequences across GPUs
- Implement the Ring Attention algorithm with online softmax
- Handle causal masking across sequence chunks

Background

Ring Attention organizes GPUs in a ring. Each GPU holds Q/K/V for its chunk of the sequence. In each step:

1. Compute attention between local Q and current K/V chunk
2. Rotate K/V to the next GPU in the ring
3. Accumulate results using the **online softmax** trick

After CP steps, every query has attended to all key/value positions.

Causal masking across chunks:

- $source_rank < cp_rank$: K/V is in the past → **no masking** needed
- $source_rank = cp_rank$: Same chunk → **standard causal mask**
- $source_rank > cp_rank$: K/V is in the future → **mask everything**

Online softmax (same trick as FlashAttention):

Maintain running max (m) and sum of exponentials (l). For each new K/V block: update m , rescale old output, add new contribution. This avoids materializing the full $T \times T$ attention matrix.

Tasks

4.1 Implement the ring attention core in `parallel/context.py`:

Inside the ring loop, you need to:

- Compute attention scores: $QK^T \times scale$
- Apply the correct causal mask based on `source_rank` vs `cp_rank`
- Update online softmax accumulators (m , l , output)

4.2 Implement `apply_context_parallelism()` in `model.py`:

- Set `cp_group` on each attention module (the forward already dispatches to ring attention)

4.3 Test with extended sequence length:

```
torchrun --standalone --nproc_per_node=2 tests/test_cp.py
torchrun --standalone --nproc_per_node=8 train.py \
    --cp_size 2 --block_size 2048 --max_steps 50
```

Demonstrate that per-GPU memory stays constant as sequence length scales with more GPUs.

The `_ring_rotate` Function (Provided)

The K/V rotation is provided. It uses `batch_isend_irecv` for non-blocking P2P communication:

```
def _ring_rotate(k, v, cp_group):  
    # Sends KV to next rank, receives from previous rank  
    # Uses dist.P2POp + dist.batch_isend_irecv  
    # Returns new k, v tensors
```

Grading

Criteria	Points
Ring attention core computation (scores + online softmax)	10
Correct causal masking across chunks	5
Integration and analysis	5

6 Part 5: Expert Parallelism

[25 points]

Learning Objectives

- Understand MoE architecture: routing, expert computation, weighted combination
- Implement top-k routing with load-balancing auxiliary loss
- Implement all-to-all token dispatch and combine for distributed experts

Background

In Mixture of Experts, selected transformer blocks use N expert FFNs instead of one:

1. **Router:** Computes gating scores, selects top- k experts per token
2. **Dispatch:** Tokens are sent to the GPU holding their assigned expert
3. **Compute:** Each GPU runs received tokens through its local experts
4. **Combine:** Results are sent back and weighted by gating scores

Expert Parallelism distributes experts across GPUs. With 8 experts and $EP = 4$, each GPU holds 2 experts. The dispatch and combine steps use **all-to-all** communication.

The **auxiliary load-balancing loss** prevents routing collapse:

$$L_{aux} = N \cdot \sum_{i=1}^N f_i \cdot P_i$$

where f_i = fraction of tokens routed to expert i , and P_i = mean routing probability for expert i .

Tasks

5.1 Implement `TopKRouter.forward()` in `parallel/expert.py`:

- Compute router logits and softmax probabilities
- Select top- k experts per token via `torch.topk()`
- Renormalize selected weights to sum to 1
- Compute auxiliary load-balancing loss

5.2 Implement `_ep_forward()` in `MoELayer` (`parallel/expert.py`):

- Determine destination rank for each token: `dest = expert_id // experts_per_rank`
- Sort tokens by destination, compute send/recv counts
- All-to-all dispatch: `dist.all_to_all_single()`
- Process received tokens through local experts
- All-to-all combine (reverse): send results back to originating ranks
- Unsort and apply gating weights

5.3 Implement `apply_moe()` in `model.py`:

- Replace MLP with `MoELayer` in every other block (layers 1, 3, 5, 7, 9, 11)

5.4 Test and analyze:

```
torchrun --standalone --nproc_per_node=2 tests/test_ep.py
# EP=4 + DP=2 with 8 experts
torchrun --standalone --nproc_per_node=8 train.py \
```

```
--ep_size 4 --num_experts 8 --max_steps 50
```

Verify expert utilization is roughly balanced (no expert collapse).

Grading

Criteria	Points
TopKRouter with correct routing and aux loss	8
All-to-all dispatch and combine	12
Integration (<code>apply_moe</code>) and analysis	5

7 Part 6: Compose 5D Parallelism

[20 points]

Learning Objectives

- Understand how 5 parallelism dimensions interact and constrain each other
- Implement multi-dimensional process group creation
- Analyze tradeoffs between different parallelism configurations

Background

The key constraint: $TP \times PP \times CP \times EP \times DP = \text{world_size}$ (8)

Process groups organize ranks into a 5D grid. The convention (outermost to innermost): $DP > PP > CP > EP > TP$. TP is innermost because it requires the highest bandwidth (NVLink).

Example with $TP = 2$, $PP = 2$, $DP = 2$ on 8 GPUs:

```
Rank grid: DP0: [[0,1],[2,3]] DP1: [[4,5],[6,7]]
TP groups: {0,1} {2,3} {4,5} {6,7}
PP groups: {0,2} {1,3} {4,6} {5,7}
DP groups: {0,4} {1,5} {2,6} {3,7}
```

Tasks

6.1 Implement `create_process_groups()` in `utils.py`:

- Create a 5D rank tensor: `torch.arange(world_size).reshape(dp, pp, cp, ep, tp)`
- For each dimension, iterate over all other indices and create `dist.new_group()`
- Return the group that this rank belongs to for each dimension

6.2 Run at least 3 different configurations and compare:

```
# Config A: TP=2, PP=2, EP=2, DP=1 (with MoE)
torchrun --standalone --nproc_per_node=8 train.py \
    --tp_size 2 --pp_size 2 --ep_size 2 --num_experts 8 --max_steps 50

# Config B: TP=2, CP=2, DP=2
torchrun --standalone --nproc_per_node=8 train.py \
    --tp_size 2 --cp_size 2 --max_steps 50

# Config C: PP=2, EP=4, DP=1 (with MoE)
torchrun --standalone --nproc_per_node=8 train.py \
    --pp_size 2 --ep_size 4 --num_experts 8 --max_steps 50
```

For each: report throughput, peak per-GPU memory, and loss at step 50.

6.3 Written analysis:

- Which configuration gives the highest throughput? Why?
- Which gives the lowest per-GPU memory? Why?
- What are the tradeoffs between TP (high bandwidth) and PP (sequential bubbles)?
- When would you add CP vs EP vs more DP ?

Grading

Criteria	Points
<code>create_process_groups()</code> correct implementation	10
3+ configurations run successfully with metrics	5
Written analysis of tradeoffs	5

Solution Reference

Complete solution files are provided in the `solutions/` directory of the repository. The solution directory mirrors the skeleton structure:

```
solutions/
+-- model.py           # apply_tensor_parallelism, apply_moe, etc.
+-- utils.py           # create_process_groups
+-- train_solution.py  # Self-contained training script
+-- parallel/
    +-- tensor.py       # ColumnParallelLinear, RowParallelLinear
    +-- pipeline.py     # create_pipeline_stages, pipeline_schedule_1f1b
    +-- context.py      # ring_attention_forward (Ring Attention)
    +-- expert.py       # TopKRouter, MoELayer with EP
```

All solutions have been verified on $8 \times$ H200 SXM GPUs with the following configurations:

- DP=8 baseline: loss ≈ 10.98 , $\sim 1.5\text{M}$ tok/sec
- TP=2 + DP=4: loss ≈ 10.98 , $\sim 744\text{K}$ tok/sec
- PP=2 + DP=4: loss ≈ 5.49 , $\sim 500\text{K}$ tok/sec
- CP=2 + DP=4: loss ≈ 10.98 , $\sim 197\text{K}$ tok/sec
- EP=4 + DP=2 (MoE): loss ≈ 11.10 , $\sim 160\text{K}$ tok/sec
- 5D (TP=2, PP=2, EP=2): loss ≈ 5.49 , $\sim 129\text{K}$ tok/sec

A RunPod Setup Guide

Recommended Pod: 8× H100 SXM 80GB (or 8× A100 SXM 80GB for budget)

```
# 1. Create a RunPod pod with 8 GPUs
#   Template: PyTorch 2.4+ with CUDA 12.x
#   Disk: 200GB+ for dataset

# 2. Verify GPU setup
nvidia-smi
nvidia-smi topo -m    # Verify NVLink connectivity

# 3. Install dependencies
pip install tiktoken datasets tqdm

# 4. Clone and set up
git clone https://github.com/VizuaruAI/vizuaru-5d-parallelism-workshop.git \
    vizz_slim && cd vizz_slim
python fineweb.py      # Download and tokenize data (~30-60 min)

# 5. Verify baseline (single GPU)
python train.py --max_steps 10

# 6. Verify DDP (all GPUs)
torchrun --standalone --nproc_per_node=8 train.py --max_steps 10
```

Important:

- Must use **SXM** GPUs (not PCIe) for NVLink, which is essential for tensor parallelism performance
- Ensure at least 200GB disk space for the FineWeb-Edu dataset
- The PyTorch template should include NCCL for GPU communication

B Key PyTorch Distributed APIs

```
# Process groups
group = dist.new_group(ranks=[0, 1, 2, 3])

# Collective operations
dist.all_reduce(tensor, op=dist.ReduceOp.SUM, group=group)
dist.all_gather(tensor_list, tensor, group=group)
dist.all_to_all_single(output, input, out_splits, in_splits, group=group)

# Point-to-point (blocking)
dist.send(tensor, dst=rank)
dist.recv(tensor, src=rank)

# Point-to-point (non-blocking, batch)
p2p_ops = [
    dist.P2POp(dist.isend, send_tensor, peer_rank),
    dist.P2POp(dist.irecv, recv_tensor, peer_rank),
]
```

```
reqs = dist.batch_isend_irecv(p2p_ops)
for req in reqs:
    req.wait()
```

C Grading Summary

Part	Points
Part 0: Setup & Baseline	10
Part 1: Data Parallelism Deep Dive	15
Part 2: Tensor Parallelism	25
Part 3: Pipeline Parallelism	25
Part 4: Context Parallelism	20
Part 5: Expert Parallelism	25
Part 6: Compose 5D Parallelism	20
Total	140